

Obrona z baz danych - material do nauki

Gotowa sciaga: co powiedziec na obronie z baz danych

Projekt: aplikacja Laravel do rozliczania wspolnych wydatkow w grupach

1. Krotka wypowiedz na poczatek

Moj projekt to aplikacja webowa do rozliczania wspolnych wydatkow w grupach, na przyklad podczas wycieczki. Uzytkownik moze utworzyc grupe, dodac do niej innych uzytkownikow, wpisywac rachunki i sprawdzac, kto ile zaplacil oraz kto ile powinien oddac.

Baza danych jest relacyjna. Najwazniejsze tabele to **users**, **groups**, **group_user**, **bills**, **bill_items**, **bill_item_user** oraz **bill_splits**. Dane sa podzielone wedlug znaczenia: osobno przechowywani sa uzytkownicy, grupy, rachunki, pozycje rachunkow i podzial kosztow.

W projekcie wykorzystuje klucze glowne, klucze obce, relacje jeden-do-wielu oraz wiele-do-wielu. Dodatkowo dla MySQL przygotowane sa triggery i funkcja skladowana, ktore pomagaja utrzymac spojnosć danych i obliczac saldo uzytkownika.

2. Co robi aplikacja?

Aplikacja pozwala:

- rejestrowac i logowac uzytkownikow,
- tworzyc grupy rozliczeniowe,
- dodawac czlonkow do grupy,
- dodawac rachunki do grupy,
- okreslac, kto zaplacil rachunek,
- dzielic koszt rachunku miedzy czlonkow grupy,
- dodawac pozycje z paragonu,
- sprawdzac saldo kazdej osoby,
- rozrozniac role **user** i **admin**.

Przykladowy scenariusz:

1. Oliwia tworzy grupe "Wycieczka w gory 2026".
2. Dodaje do grupy Adama i Ewe.
3. Oliwia placi rachunek 600 zl za obiad.
4. System dzieli rachunek po 200 zl na trzy osoby.
5. Oliwia jest 400 zl na plusie, a Adam i Ewa po 200 zl na minusie.

3. Najwazniejsze tabele

users

Tabela **users** przechowuje uzytkownikow systemu.

Najwazniejsze kolumny:

- **id** - klucz glowny,
- **name** - imie lub nazwa uzytkownika,
- **email** - adres e-mail,
- **password** - zahashowane haslo,
- **role** - rola uzytkownika, na przyklad **user** albo **admin**.

Co powiedziec:

Tabela **users** przechowuje dane uzytkownikow. Kazdy uzytkownik ma unikalny identyfikator **id**. Dodalem tez kolumnę **role**, zeby rozrozniac zwyklego uzytkownika i administratora.

groups

Tabela **groups** przechowuje grupy rozliczeniowe.

Najwazniejsze kolumny:

- **id** - klucz glowny,
- **name** - nazwa grupy,
- **owner_id** - wlasciciel grupy, klucz obcy do **users.id**,
- **total_amount** - laczna suma wydatkow w grupie.

Co powiedziec:

Tabela **groups** przechowuje grupy, w ktorych rozliczane sa wydatki. Kazda grupa ma wlasciciela wskazanego przez **owner_id**. Pole **total_amount** przechowuje sume wszystkich rachunkow w grupie.

Relacja:

users 1 --- N groups

Jeden uzytkownik moze byc wlascicielem wielu grup.

group_user

Tabela **group_user** jest tabela posrednia miedzy **users** i **groups**.

Najwazniejsze kolumny:

- **id** - klucz glowny,
- **group_id** - klucz obcy do **groups.id**,
- **user_id** - klucz obcy do **users.id**.

Co powiedziec:

Ta tabela realizuje relacje wiele-do-wielu. Jeden uzytkownik moze nalezec do wielu grup, a jedna grupa moze miec wielu uzytkownikow.

Relacja:

users N --- M groups

bills

Tabela **bills** przechowuje rachunki lub wydatki.

Najwazniejsze kolumny:

- **id** - klucz glowny,
- **group_id** - grupa, do ktorej nalezy rachunek,
- **payer_id** - uzytkownik, ktory zaplacil rachunek,
- **description** - opis wydatku,
- **amount** - kwota rachunku,
- **date** - data rachunku.

Co powiedziec:

Tabela **bills** przechowuje konkretne wydatki. Kazdy rachunek nalezy do jednej grupy przez **group_id** i ma platnika wskazanego przez **payer_id**.

Relacje:

groups 1 --- N bills
users 1 --- N bills jako platnik

bill_items

Tabela **bill_items** przechowuje pozycje z rachunku lub paragonu.

Najwazniejsze kolumny:

- **id** - klucz glowny,
- **bill_id** - rachunek, do ktorego nalezy pozycja,
- **name** - nazwa pozycji,
- **price** - cena,
- **quantity** - ilosc.

Co powiedziec:

Tabela **bill_items** pozwala rozbic rachunek na konkretne pozycje, na przyklad pizza, napoje albo paliwo. Jeden rachunek moze miec wiele pozycji.

Relacja:

bills 1 --- N bill_items

bill_item_user

Tabela **bill_item_user** laczy pozycje rachunku z uzytkownikami.

Co powiedziec:

Ta tabela wskazuje, ktorzy uzytkownicy odpowiadaja za konkretna pozycje z paragonu. Jest to relacja wiele-do-wielu, bo jedna pozycja moze dotyczyc wielu osob, a jeden uzytkownik moze miec wiele pozycji.

Relacja:

bill_items N --- M users

bill_splits

Tabela **bill_splits** przechowuje podzial kosztow rachunku.

Najwazniejsze kolumny:

- **id** - klucz glowny,
- **bill_id** - rachunek,
- **user_id** - uzytkownik,
- **amount** - kwota przypisana do uzytkownika,
- **is_paid** - informacja, czy dana czesc jest oplacona.

Co powiedziec:

Tabela **bill_splits** przechowuje, ile dana osoba powinna zaplacic za konkretny rachunek. Dzieki temu mozna pozniej obliczyc saldo kazdego czlonka grupy.

4. Schemat relacji

Uproszczony schemat:

```
users
| 1:N przez owner_id
groups
| 1:N
bills
| 1:N
bill_items

users N:M groups przez group_user
users N:M bill_items przez bill_item_user
bills 1:N bill_splits
users 1:N bill_splits
```

Najwazniejsze klucze obce:

- **groups.owner_id** -> **users.id**,
- **group_user.group_id** -> **groups.id**,
- **group_user.user_id** -> **users.id**,
- **bills.group_id** -> **groups.id**,

- **bills.payer_id -> users.id,**
- **bill_items.bill_id -> bills.id,**
- **bill_item_user.bill_item_id -> bill_items.id,**
- **bill_item_user.user_id -> users.id,**
- **bill_splits.bill_id -> bills.id,**
- **bill_splits.user_id -> users.id.**

Co powiedziec:

Klucze obce zapewniaja spójność danych. Na przykład nie powinien powstac rachunek przypisany do grupy, która nie istnieje. W migracjach zastosowalem **onDelete('cascade')**, wiec po usunieciu rekordu nadrzednego powiazane dane sa usuwane razem z nim.

5. Jak dziala dodanie rachunku?

Proces:

1. Uzytkownik wybiera grupe.
2. Podaje opis rachunku, kwote i platnika.
3. Aplikacja sprawdza, czy platnik nalezy do grupy.
4. Tworzony jest rekord w tabeli **bills**.
5. System dzieli kwote rachunku miedzy czlonkow grupy.
6. Podzial zapisywany jest w **bill_splits**.
7. Aktualizowana jest suma wydatkow grupy **total_amount**.

Co powiedziec:

Po dodaniu rachunku system najpierw waliduje dane, a potem sprawdza, czy platnik jest czlonkiem grupy. Nastepnie tworzy rachunek i automatycznie tworzy wpisy w **bill_splits**, dzielac kwote po rowno miedzy czlonkow grupy.

6. Jak liczone jest saldo?

Saldo liczone jest wedlug wzoru:

$\text{saldo} = \text{suma zaplaconych rachunkow} - \text{suma naleznosci z bill_splits}$

Przyklad:

Oliwia zaplacila 600 zl za rachunek. W grupie sa 3 osoby, wiec kazda osoba ma udzial 200 zl.

Oliwia: $600 - 200 = 400$
 Adam: $0 - 200 = -200$
 Ewa: $0 - 200 = -200$

Co powiedziec:

Jezeli saldo jest dodatnie, to znaczy, ze uzytkownik zaplacyl wiecej niz powinien i inni powinni mu oddac pieniadze. Jezeli saldo jest ujemne, to znaczy, ze uzytkownik powinien oddac pieniadze.

7. Normalizacja bazy

Co powiedziec:

Baza jest znormalizowana, bo dane sa rozdzielone na osobne tabele wedlug ich znaczenia. Uzytkownicy sa w tabeli **users**, grupy w **groups**, rachunki w **bills**, pozycje rachunkow w **bill_items**, a podzial kosztow w **bill_splits**. Dzieki temu unikam powtarzania danych i latwiej utrzymac spojność.

Przyklad:

Nie zapisuje listy uzytkownikow bezposrednio w tabeli **groups** jako tekst. Zamiast tego mam tabele **users** oraz tabele posrednia **group_user**. To jest poprawne podejscie relacyjne.

8. Triggery w MySQL

W projekcie sa przygotowane triggery dla MySQL.

Triggery aktualizujace sume grupy

Triggery:

- **update_group_total_after_bill_insert**,
- **update_group_total_after_bill_update**,
- **update_group_total_after_bill_delete**.

Co robia:

- po dodaniu rachunku zwiekszaja **groups.total_amount**,
- po edycji rachunku aktualizuja **groups.total_amount**,
- po usunieciu rachunku zmniejszaja **groups.total_amount**.

Co powiedziec:

Triggery automatycznie pilnuja sumy wydatkow w grupie. Dzieki temu **total_amount** jest aktualizowane na poziomie bazy danych, a nie tylko w kodzie aplikacji.

Trigger walidacyjny

Trigger:

- **validate_user_in_group_before_item_assign**.

Co robi:

Ten trigger sprawdza, czy uzytkownik przypisywany do pozycji rachunku nalezy do grupy, w ktorej znajduje sie rachunek. Jezeli nie nalezy, baza blokuje taki zapis.

Dlaczego to jest wazne:

To zabezpiecza integralnosc danych. Nawet jesli ktos probowalby ominac interfejs aplikacji, baza danych nie pozwoli przypisac pozycji rachunku osobie spoza grupy.

9. Funkcja składowana

Funkcja:

```
get_user_net_balance(user_id, group_id)
```

Co robi:

Funkcja składowana oblicza saldo użytkownika w danej grupie. Sumuje rachunki zapłacone przez użytkownika, potem odejmuje jego należności z tabeli **bill_splits**.

Wzor:

suma zapłaconych rachunków - suma należności = saldo

Co powiedzieć:

Funkcja składowana pokazuje, że część logiki można przenieść do bazy danych. W tym projekcie funkcja pomaga obliczyć saldo netto użytkownika dla konkretnej grupy.

10. SQLite a MySQL

Co powiedzieć, jeśli zapytają:

Domyslnie projekt może działać na SQLite, bo jest to proste przy uruchamianiu lokalnym. Natomiast triggery i funkcja składowana są przygotowane dla MySQL. Jeżeli aplikacja działa na SQLite, to część logiki, na przykład aktualizacja **total_amount**, wykonywana jest w PHP. Na MySQL ta logika może być obsługiwana przez triggery i funkcje składowane.

11. Przykładowe zapytania SQL do pokazania

Pokaz wszystkich użytkowników

```
SELECT id, name, email, role  
FROM users;
```

Pokaz grupy razem z właścicielem

```
SELECT g.id, g.name, u.name AS owner_name, g.total_amount  
FROM groups g  
JOIN users u ON u.id = g.owner_id;
```

Pokaz członków konkretnej grupy

```
SELECT u.id, u.name, u.email  
FROM users u  
JOIN group_user gu ON gu.user_id = u.id  
WHERE gu.group_id = 1;
```

Pokaz rachunki w grupie

```
SELECT b.description, b.amount, b.date, u.name AS payer  
FROM bills b  
JOIN users u ON u.id = b.payer_id  
WHERE b.group_id = 1;
```

Oblicz sume rachunkow w grupie

```
SELECT group_id, SUM(amount) AS total
FROM bills
GROUP BY group_id;
```

Pokaz podzial kosztow

```
SELECT b.description, u.name, bs.amount, bs.is_paid
FROM bill_splits bs
JOIN bills b ON b.id = bs.bill_id
JOIN users u ON u.id = bs.user_id
WHERE b.group_id = 1;
```

Oblicz saldo uzytkownika recznie

```
SELECT
    u.name,
    COALESCE(paid.total_paid, 0) - COALESCE(owed.total_owed, 0) AS balance
FROM users u
LEFT JOIN (
    SELECT payer_id, SUM(amount) AS total_paid
    FROM bills
    WHERE group_id = 1
    GROUP BY payer_id
) paid ON paid.payer_id = u.id
LEFT JOIN (
    SELECT bs.user_id, SUM(bs.amount) AS total_owed
    FROM bill_splits bs
    JOIN bills b ON b.id = bs.bill_id
    WHERE b.group_id = 1
    GROUP BY bs.user_id
) owed ON owed.user_id = u.id;
```

12. Pytania, ktore moga pasc na obronie

Co to jest klucz glowny?

Klucz glowny to kolumna, ktora jednoznacznie identyfikuje rekord w tabeli. W moim projekcie wiekszosc tabel ma klucz glowny **id**.

Co to jest klucz obcy?

Klucz obcy to kolumna, ktora wskazuje na klucz glowny w innej tabeli. Na przyklad **bills.group_id** wskazuje na **groups.id**.

Jaka jest roznica miedzy relacja jeden-do-wielu a wiele-do-wielu?

Relacja jeden-do-wielu oznacza, ze jeden rekord z pierwszej tabeli moze miec wiele powiazanych rekordow w drugiej tabeli. Na przyklad jedna grupa ma wiele rachunkow. Relacja wiele-do-wielu oznacza, ze wiele rekordow z jednej tabeli moze laczy sie z wieloma rekordami z drugiej tabeli. Na przyklad wielu uzytkownikow moze nalezec do wielu grup.

Po co jest tabela group_user?

Tabela **group_user** jest potrzebna do relacji wiele-do-wielu miedzy uzytkownikami i grupami.

Po co jest tabela bill_splits?

Tabela **bill_splits** przechowuje informacje, ile każdy użytkownik powinien zapłacić za dany rachunek.

Co oznacza payer_id?

payer_id wskazuje użytkownika, który faktycznie zapłacił rachunek.

Jak zabezpieczona jest spójność danych?

Spójność danych jest zabezpieczona przez klucze obce, walidacje w aplikacji oraz triggery w MySQL.

Co robi onDelete('cascade')?

Powoduje automatyczne usunięcie danych powiązanych. Na przykład po usunięciu grupy mogą zostać usunięte jej rachunki i powiązania z użytkownikami.

Czy baza jest znormalizowana?

Tak. Dane są rozdzielone na osobne tabele, co ogranicza powtarzanie informacji i ułatwia utrzymanie bazy.

Po co są triggery?

Triggery automatycznie wykonują logikę po określonych operacjach w bazie. W moim projekcie aktualizują łączną kwotę wydatków grupy i sprawdzają, czy użytkownik przypisywany do pozycji rachunku należy do grupy.

Po co jest funkcja składowana?

Funkcja składowana **get_user_net_balance** oblicza saldo użytkownika w grupie bezpośrednio w bazie danych.

13. Najkrótsza wersja do nauczenia na pamięć

Mój projekt to aplikacja do rozliczania wspólnych wydatków w grupach. Baza danych składa się z tabel odpowiedzialnych za użytkowników, grupy, rachunki, pozycje rachunków i podział kosztów. Relacje są realizowane przez klucze obce oraz tabele pośrednie, na przykład **group_user** dla relacji wiele-do-wielu między użytkownikami i grupami. Po dodaniu rachunku aplikacja dzieli koszt między członków grupy i zapisuje ten podział w **bill_splits**. Saldo użytkownika liczone jest jako suma zapłaconych rachunków minus suma jego należności. Dodatkowo dla MySQL przygotowałem triggery oraz funkcję składowaną, które pomagają utrzymać spójność danych i obliczać saldo.

14. Co pokazać w aplikacji podczas obrony?

1. Logowanie na konto demo.
2. Wejście do grupy "Wycieczka w góry 2026".
3. Pokazanie członków grupy.
4. Pokazanie rachunku 600 zł.
5. Wyjaśnienie, że koszt jest dzielony po 200 zł na osobę.
6. Pokazanie sald: osoba płacąca jest na plusie, pozostali na minusie.

7. Dodanie nowego rachunku i pokazanie, że suma grupy oraz salda się zmieniają.
8. Wspomnienie, że admin ma dodatkowe uprawnienia przez pole **role**.

15. Na co uważać

- Nie mów tylko "mam tabele users i groups" - zawsze dopowiedz, po co one są.
- Przy relacji wiele-do-wielu od razu podaj tabele pośrednie.
- Przy saldzie pamiętaj wzór: zapłacone minus należne.
- Przy triggerach zaznacz, że są przygotowane dla MySQL.
- Jeśli zapytają o SQLite, powiedz, że lokalnie ułatwia uruchomienie, ale logika triggerów jest dla MySQL.